ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/350704479

Efficient Trajectory Contact Query Processing

Chapter · April 2021

DOI: 10.1007/978-3-030-73194-6_44

CITATIONS
8

READS
131

5 authors, including:



Pingfu Chao
Soochow University (PRC)
46 PUBLICATIONS **221** CITATIONS
SEE PROFILE



Lei Li
The Hong Kong University of Science and Technology (Guangzhou)
39 PUBLICATIONS **413** CITATIONS
SEE PROFILE

All content following this page was uploaded by Pingfu Chao on 15 February 2022.

The user has requested enhancement of the downloaded file.

Efficient Trajectory Contact Query Processing

Pingfu Chao[1], Dan He[1], Lei Li[1], Mengxuan Zhang[1], and Xiaofang Zhou[1]

School of Information Technology and Electrical Engineering,
University of Queensland, Australia
{p.chao,d.he,l.li3}@uq.edu.au, mengxuan.zhang@uqconnect.edu.au,
zxf@itee.uq.edu.au

Abstract. During an infectious disease outbreak, the contact tracing is regarded as the most crucial and effective way of disease control. As the users' trajectories are widely obtainable due to the ubiquity of positioning devices, the contact tracing can be achieved by examining trajectories of confirmed patients to identify other trajectories that are contacted either directly or indirectly. In this paper, we propose a generalised Trajectory Contact Search (TCS) query, which models the contact tracing problem as well as other similar trajectory-based problems. In addition, we answer the query by proposing an iterative algorithm that finds contacted trajectories progressively along the transmission chains, and we further optimise each iteration in terms of time and space efficiency by proposing a hop scanning algorithm and a grid-based time interval tree. Extensive experiments on large-scale real-world data demonstrate the effectiveness of our proposed solutions over baseline algorithms.

1 Introduction

Nowadays, our daily movements are collected into trajectory data by various devices like GPS, Bluetooth, cellular tower, etc. Among different trajectory query types, there is a lack of research on exploiting the trajectory's *contact* feature, which happens when two objects travel nearby each other over a period of time, such that information (contagious disease, chemical/radiative leaks, airborne materials, etc.) could spread from one to another. A direct application is the contact tracing during the COVID-19 pandemic. Massive literature emphasises the importance of contact tracing and several countries have established apps to track contact events through Bluetooth connection between devices. However, such a method is hardly practical as it requires users' strict participation and it is not capable of detecting contact events with different radius due to the hardware limitation. In contrast, contact tracing through trajectories is more applicable as the trajectory distance is measurable and the data is widely available.

Therefore, in this paper, we propose a new type of trajectory query, termed as *Trajectory Contact Search (TCS)*, which finds all trajectories that directly and indirectly contact the query trajectory. Specifically, when two trajectories appear within a distance over a time period, we say they make a *contact* and one can *influence* the other. Then, given a query trajectory Tr_q, a distance ϵ, and the time step threshold k, we aim to find not only all trajectories R' it contacts, but also all the trajectories contacted by the influenced results R' subsequently.

In fact, trajectory contact tracing is non-trivial. As pre-computing all contact events is not viable due to the flexibility of contact definition (ϵ and k), the query can only be answered by searching direct contacts to the influenced trajectory recursively. Besides, as a contact event requires both spatial and temporal continuity, new index and scanning algorithm are required to store and retrieve timestamp-level trajectories efficiently. Overall, our contributions are as follows:

- We propose a *Trajectory Contact Search* query for contact tracing problem.
- We propose an iteration-based solution to answer the contact search query without redundancy. In addition, we propose a *hop scanning* algorithm and a *time interval grid* index to further improve the time and space efficiency.
- Extensive experiments on large-scale real-world dataset show that our methods can answer the *TCS* query more efficiently than existing methods.

2 Related Works

To the best of our knowledge, the *travelling group discovery* problem is closely related to *TCS*. It finds all groups of objects that move together over a period of time. Depending on how to define proximity (*distance-based* [1, 2, 4], or *density-based* [3, 7, 9]) and whether the time period is required to be consecutive [2, 3, 7] or not [5, 9], various group patterns are identified. To discover the groups, the trajectories are first sliced into temporal snapshots, then a clustering algorithm or predefined criteria is applied to each snapshot to find groups. Finally, the clusters from adjacent snapshots are intersected and concatenated until forming a long time sequence satisfying travel requirements. Besides, to enable distance comparison in every timestamp, linear interpolation is introduced to ensure an object to appear in every snapshot it crosses, which greatly inflates the input size and the processing cost. To reduce the cost, [3] uses trajectory segments instead of points, and it further simplifies trajectories using *Douglas-Peucker* algorithm. Meanwhile, [7] proposes a *travelling buddy* structure to capture the minimal groups of objects and perform intersection on buddies instead.

Besides, *IMO* [8] is the only demo that works on a similar problem to ours. However, it focuses on simulating the disease spread and analysing the effectiveness of the policies in preventing transmission. The demo is integrated into an open-source spatial-temporal database and utilises the built-in spatial-temporal indices to reduce the search space of finding the infected trajectories. Although it is regarded as the first work that introduces the trajectory contact tracing problem, it doesn't work on the query optimisation, which is our main focus.

3 Problem Statement

Definition 1 (Trajectory). *A trajectory is a series of chronologically ordered points* $Tr_o = \langle p_1 \rightarrow p_2 \rightarrow \cdots \rightarrow p_n \rangle$ *representing the historical trace of an object* o*. Each point* $p_i = \langle x, y, t \rangle$ *indicates the location of* o *at time* $p_i.t$*.*

Note that, since the trajectories are sampled periodically, we define that the time interval between any two consecutive trajectory points is the multitude of Δt, i.e. $p_{n+1}.t - p_n.t = x * \Delta t, x \in \mathcal{N}$, which we call a *time step*.

As for a *contact event*, two objects are defined as contacted if their trajectories (1) are close to each other at a certain point in time, and (2) such proximity is kept for a continuous period of time, formally defined as follows:

Definition 2 (Contact Event). *Given a distance threshold ϵ and a duration k, objects a and b are directly contacted during $[t_u, t_v]$ if $\forall t_i \in [t_u, t_v], dist(a, b, t_i) \leq \epsilon$ and $t_v - t_u \geq k * \Delta t$, denoted as a contact event $C_{\epsilon,k}(a, b, [t_u, t_v])$.*

Subsequently, we define the *direct contact search* problem below:

Definition 3 (Direct Contact Search (DCS)). *Given a trajectory set R, a query trajectory Tr_q, a starting time t, a distance threshold ϵ and a duration k, a direct contact search $DCS(Tr_q, t, \epsilon, k)$ returns all trajectories Tr_o that satisfies: $\exists C_{\epsilon,k}(q, o, [t_u, t_v])$ where $t_u \geq t$ (direct contact).*

Note that, if not specified, the query starting time t is assumed to set to the starting time of Tr_q. Now we are ready to define the *trajectory contact search* which further capture the indirect contacts:

Definition 4 (Trajectory Contact Search (TCS)). *Given a trajectory set R, a query trajectory Tr_q, a distance threshold ϵ and a duration k, the trajectory contact search $TCS(Tr_q, \epsilon, k)$ returns all trajectories Tr_a which satisfy: there exists a sequence of trajectories $\langle Tr_0, Tr_1, ..., Tr_n \rangle$ where (1) $Tr_0 = Tr_q$, $Tr_n = Tr_a$, (2) $\forall i \in [1, n]$, Tr_i and Tr_{i-1} are contacted directly as $C_{\epsilon,k}(i-1, i, [ct_i, ct_i + k * \Delta t])$ and (3) $\forall i \in [2, n], ct_i \geq ct_{i-1} + k * \Delta t$.*

4 Iteration-Based Trajectory Contact Search

A direct solution to address *TCS* follows the same routine of the disease transmission process. Starting from the query trajectory, it performs a *DCS* on a contacted trajectory in each iteration. Then trajectories retrieved by *DCS* are regarded as the newly contacted trajectories. The algorithm terminates when all contacted trajectories are examined. Intuitively, the iteration process may follow either *Breath-First Search (BFS)* or *Depth-First Search (DFS)* order. However, both can retrieve the result correctly but they may incur redundant computation, as one trajectory may contact multiple reported trajectories. Motivated by this, we propose an *Iterative Direct Contact Search (Iterative-DCS)* algorithm that performs iterations through temporal order, which ensures that each contacted trajectory is observed and processed only once, demonstrated in Algorithm 1.

To enable the distance calculation between trajectories all the time, in the preprocessing step, we align all trajectories to the same frequency through linear interpolation, such that each trajectory has one point every time step Δt during its lifetime. In the algorithm, we maintain a priority queue Q_T that stores all contacted trajectories that have not been processed, where each entry consists of a contacted trajectory Tr_i and a time t_i indicating the time that this trajectory was first contacted, and the priority is defined by t_i of each entry. Initially, the given query trajectory Tr_q and its trajectory starting time are enqueued into

Algorithm 1: Iterative-DCS Algorithm

Input: Trajectory Set R, Query Trajectory Tr_q, Contact Parameter $\langle \epsilon, k \rangle$
Output: A set of contacted trajectories R_q

```
1  Let Q_T be a priority queue to store contacted trajectory;
2  Q_T.enqueue(⟨Tr_q, t_q⟩);
3  while Q_T ≠ ∅ do
4      ⟨Tr_i, t_i⟩ ← Q_T.dequeue() ;
5      L ← DCS(Tr_i, t_i, ϵ, k) ;
6      for ⟨Tr_j, t_j⟩ ∈ L and Tr_j ∉ R_q do
7          if ⟨Tr_j, t'_j⟩ ∈ Q_T and t_j < t'_j then
8              Update the entry of Tr_j with t_j;
9          else if ⟨Tr_j, t'_j⟩ ∉ Q_T then
10             Q_T.enqueue(⟨Tr_j, t_j⟩);
11     R_q ← R_q ∪ {Tr_i} ;
12 return R_q;
```

Q_T (Line 2). In each iteration, we dequeue a trajectory from Q_T with smallest t_i and perform a *DCS* to retrieve a list L of contacted trajectories (Line 3-5). We then check if each item already exists in Q_T. If yes, we update the entry of Tr_j only if t_j is earlier than the existing contact time (Line 7-8). Otherwise, we enqueue the pair to Q_T (line 9-10) directly. Lastly, we add it to the result set R_q (Line 11). The algorithm terminates when Q_T is empty and we return R_q as the *TCS* result (Line 12). By doing so, the *Interactive-DCS* guarantees that each contacted trajectories is processed only once.

Regarding the *DCS* algorithm, the main goal is to find trajectories which have no less than k consecutive points that are pair-wisely close to Tr_q. Therefore, we employ a slice-based grid for indexing. Specifically, for each timestamp t, we build a grid index G_t for all trajectory points at t, where each point is assigned with the trajectory id it belongs to. Meanwhile, we construct a B-tree for the temporal dimension to quickly retrieve the corresponding grid slice. The *DCS* algorithm works as follows: 1) For each point $p_i \in Tr_q$ $(1 \leq i \leq n)$, we perform a spatial range query on the grid index $G_{p_i.t}$ that contains p_i to retrieve the neighbour list $N_\epsilon(p_i)$ of points whose distance to p_i is no larger than ϵ. 2) We scan through all the neighbour lists chronologically and keep track of the continuous occurrence for each trajectory by maintaining a hash table. Specifically, for a trajectory who has consecutive points appearing in a sequence of neighbour lists, we record the first and last occurrence time step, and it resets whenever there is an interruption before reaching k time steps. 3) We report all the trajectories with no less than k consecutive points falling into the neighbour lists.

5 Advanced Contact Search Algorithm

In this section, we introduce two optimisation strategies on the *DCS* algorithm to improve the time and space efficiency, respectively. Recall that during one *DCS* in the *iterative-DCS* algorithm, we generate a neighbour list for every time step of the trajectory, and scan all of them chronologically. However, it is

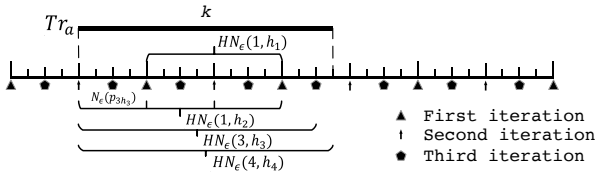


Fig. 1: Example of a hop scanning algorithm

impossible for two trajectories to be contacted during a duration of $k * \Delta t$ if they fail to keep the distance at any moment amid the period. Thus, the full scan is unnecessary, motivated by which we propose a hop scanning algorithm.

5.1 Hop Scanning Algorithm

The idea of *hop scanning* is first generating a neighbour list N_ϵ for every certain number of points, such that two adjacent neighbour lists are far enough but still within a time duration $k * \Delta t$. Therefore, if the intersection result of them is empty, we can safely prune this time period. We repeat it for the remaining time periods iteratively by shrinking the gap size until all points are investigated. Specifically, given a query trajectory Tr_q, in the ith iteration, we first generate neighbour lists (if not exist) once every h_i points, i.e., p_{jh_i} where $j = 0, 1, 2, \ldots$ and $h_i = \frac{k*\Delta t}{2^i}$, representing the current hop length. After generating the list $N_\epsilon(p_{jh_i})$, we create *hop neighbour lists*, denoted by $HN_\epsilon(j, h_i)$ to check if there is any possible contact event during this period, where $HN_\epsilon(j, h_i) = \bigcap_{l=0}^{2^i-1} N_\epsilon(p_{(j+l)h_i})$. After the intersection, objects in $HN_\epsilon(j, h_i)$ are the only objects that possibly contact with Tr_q during the period of $[jh_i, (j+1)h_i]$. Besides, given that $h_i < k * \Delta t$ holds at all time, this result can further extend to $[jh_i, jh_i + k * \Delta t]$, meaning no contact event during k.

Fig. 1 illustrates an example of hop scanning algorithm. In the figure, each line point represents a time step of the query trajectory, and $k = 16$ is given by the query. The top bold line lasting k time steps is a contact event between Tr_a and Tr_q. In the first iteration ($h_1 = 8 * \Delta t$), the time steps labelled by triangles are scanned to generate neighbour lists. When calculating the hop neighbour list, we can find that $Tr_a \in HN_\epsilon(1, h_1)$ as both $N_\epsilon(p_{h_1})$ and $N_\epsilon(p_{2h_1})$ contain it. However, from the second iteration ($h_2 = 4 * \Delta t$), the neighbour lists to be generated consist of both existing triangle ones and the new arrow-labelled ones, and the hop neighbour lists will not be built from scratch. For example, the $HN_\epsilon(1, h_2)$ is the intersection result of three components: (1) the calculated parent $HN_\epsilon(1, h_1)$, (2) $N_\epsilon(p_{3h_2})$ that falls within the time range of the parent $HN_\epsilon(1, h_1)$ and (3) an extra $N_\epsilon(p_{h_2})$ which is adjacent to $HN_\epsilon(1, h_1)$. In fact, for all subsequent hop neighbour list calculations, we will find that there is always one item in components (1) and (3), while the number of items in second component is decided by the iteration count, i.e. $2^{i-1} - 1$. Meanwhile, it is also worth noting that different hop neighbour lists may share the same parent. In general, the hop neighbour lists follow a binary tree structure, where each non-leaf node has two children (except the first and last HN in each iteration). Therefore, to avoid repetitive calculation, instead of generating hop neighbour lists sequentially, in i^{th} iteration, we first intersect each parent HN with all

$2^{i-1}-1$ neighbour lists that fall within its time range (component (2)), the result is then split into two HNs by intersecting left and right adjacent neighbour lists (component (3)), respectively. Hence, the algorithm ensures both children are generated by only processing their parent once. Meanwhile, even in the worst-case scenario when all neighbour lists are generated, each list is only generated once, which does not introduce extra cost over the full scan solution. Overall, it accelerates the process by efficiently pruning out the irrelevant time periods.

5.2 Time Interval Grid Index

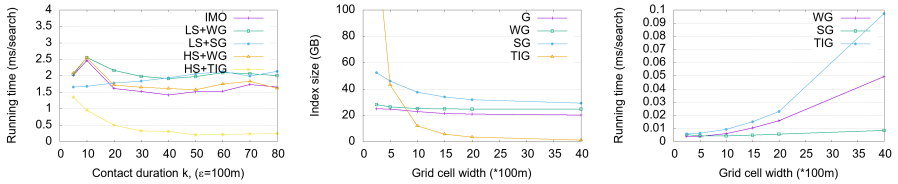
In *Iterative-DCS* algorithm, we use slice-based grid to index all trajectory points. As the slice-based grid index maintains one index per time step, it aims for the optimal efficiency at the cost of massive space consumption. Note that the number of time steps is determined by the time span of the dataset and Δt. it is usually very large. Alternatively, by fitting each slice with multiple time steps, the index size can be significantly reduced at the cost of slower query speed. Therefore, we propose the *time interval grid (TIG)* index which reduces the number of the indexed points while keeping the query efficiency. The main idea of our index is to achieve spatial partitioning using grid index while constructing a time interval tree in each grid cell to enable efficient temporal searches.

Data Structure. The data structure consists of two layers. The first layer is a spatial grid index. When constructing the index, each trajectory from the dataset is split into a set of sub-trajectories, named as trajectory fragments, each of which is the longest sequence of consecutive trajectory points that falls in the same grid cell. The second layer is a time interval index. Following the idea from [6], for each grid cell i, we build a time interval index S_i to store all fragments located in it. Each fragment $f_(j) = Tr_o(a, b)$ is represented by the timestamp of its left and right endpoints, i.e., $l_j = pa.t$ and $r_j = p_b.t$ ($l_j \leq r_j$). Therefore, a fragment f_j is stored as a time interval $[l_j, r_j]$ in S_i. The construction process is omitted due to page limit and can be found in the original paper [6].

Neighbour Search Query. To find neighbours of a given trajectory point p_q at time t_q, we first search the grid index to find all cells intersecting the circular query region centred at p_q with the radius ϵ. For each grid cell I_i, we search the corresponding time interval index S_i by performing an interval stabbing query [6]. An important feature of the tree structure is that all intervals stabbed by the query q stay close, which ensures optimal query performance. Finally, we return the ids of trajectories that are stabbed by q. After the time interval tree search in each cell, we merge the results found from different cells and filter out the points whose actual distances to the query point p_q are larger than *epsilon* due to the low grid resolution. In general, since every time we compare an interval with q during traversal, we either add it to the output (if stabbed) or stop the subsequent search, the query complexity is $O(1 + m)$ (m is the number of output intervals), and the tree construction time and space complexity are both $O(n)$ (n is the number of intervals in S). Therefore, both search and space complexities are theoretically optimal. Compared with the basic slice-based grid index, the time interval tree only stores endpoints of each trajectory fragments

Table 1: Data specification

Name	*# of Objects*	*# of Points*	*Time Span (hours)*	*Avg Length (sec)*	*Map Size (km^2)*	*Point Density ($pts/(km^2 * sec)$)*
Beijing-M	196K	6.82M	120	398	57	3.18
Beijing-L	953K	50.82M	120	469	589	1.76



(a) Hop scanning efficiency (M) **(b)** Index sizes comparison (L) **(c)** Index query efficiency (L)

Fig. 2: Experimental results

instead of one point per time step. Although one trajectory may have multiple fragments, it is still more effective in terms of the storage cost (especially when grid cells are big), while its time efficiency is competitive.

6 Experiments

6.1 Experiment Settings

There is a lack of public trajectory dataset that has sufficiently large scale as well as high density, especially for pedestrian trajectories, so we conduct our experiments on real-world commercial data sets of taxi trajectories from Beijing, China. Besides, we set $\Delta t = 1sec$, and the data specification is listed in Table 1.

To ensure the correctness of evaluation results, we randomly select 25 trajectories from dataset as the query trajectories, and evaluate the total running time as well as the average running time taken to trace one contacted object (ms/rec). Our algorithms are implemented in C++, and all experiments are conducted on a single server. Our candidate solutions include our basic *iterative-DCS* solution with Linear Scanning (**LS**) and Slice-based Grid (**SG**) index, as well as the Hop Scanning (**HS**) algorithm and the Time Interval Grid (TIG) index. In addition, we introduce two more grid-based indices as references: (1) The typical spatial grid (**G**) which partitions the trajectory points only by their spatial location. (2) The Window-based Grid (**WG**) first partitions the temporal dimension into multiple windows and constructs a grid index for each window. In addition, we implement the only related work **IMO** [8] as another baseline. We test **IMO** on different grid indices, and we find that it achieves the best search performance when using **WG** with a 100s window, which will be set as the default setting.

6.2 Effectiveness Study

Hop Scanning Algorithm. Fig. 2a depicts the average running time of each DCS search using different scanning algorithms and indices. We can see that both hop scanning solutions (HS+WG & HS+TIG) outperform their linear scanning counterpart constantly, and the gap is more significant as k increases, which meets our expectation as a longer k enables more pruning opportunities. On the other hand, our advanced algorithm HS+TIG outperforms IMO significantly.

Time Interval Grid Index. We evaluate the performance of our proposed TIG index when $\epsilon = 100m$ and $k = 30sec$. Figures 2b and 2c illustrate its space and time efficiency on **Beijing-L**. As we expected, the slice-based grid consumes the largest space, which reduces steadily as the grid size increases due to the reduction of grid cell entries. In contrast, the grid cell size is crucial to TIG. A smaller grid cell means more trajectory fragments, resulting to more index entries. The figures show that the size of TIG is only $\frac{1}{3}$ of the others once the grid width reaches 1km. Meanwhile, Fig. 2c shows that extremely small grid cells do not generally lead to better query performance. This is due to the excessive number of grid cells creates too many index entries, which slows down the entry search. In our experiments, by choosing a grid cell of 1km $\times$ 1km, we can achieve almost the best query performance and storage cost.

7 Conclusion

In this paper, we introduced a new trajectory contact search query to model the trajectory contact problem. We proposed an iteration-based algorithm which performs direct contact search iteratively to find all contacted trajectories. In addition, we further optimised it with a hop scanning algorithm and a time interval grid index, which improved the time and space efficiency, respectively. The experiments show that our proposal achieves both faster query speed and smaller space consumption.

References

1. Buchin, K., Buchin, M., van Kreveld, M., Speckmann, B., Staals, F.: Trajectory grouping structure. In: WADS. pp. 219–230. Springer (2013)
2. Gudmundsson, J., van Kreveld, M., Speckmann, B.: Efficient detection of motion patterns in spatio-temporal data sets. In: Proceedings of the 12th annual ACM international workshop on Geographic information systems. pp. 250–257 (2004)
3. Jeung, H., Shen, H.T., Zhou, X.: Convoy queries in spatio-temporal databases. In: ICDE. pp. 1457–1459. IEEE (2008)
4. van Kreveld, M., Löffler, M., Staals, F., Wiratma, L.: A refined definition for groups of moving entities and its computation. International Journal of Computational Geometry & Applications **28**(02), 181–196 (2018)
5. Li, Z., Ding, B., Han, J., Kays, R.: Swarm: Mining relaxed temporal moving object clusters. PVLDB **3**(1-2), 723–734 (2010)
6. Schmidt, J.M.: Interval stabbing problems in small integer ranges. In: International Symposium on Algorithms and Computation. pp. 163–172. Springer (2009)
7. Tang, L.A., Zheng, Y., Yuan, J., Han, J., Leung, A., Hung, C.C., Peng, W.C.: On discovery of traveling companions from streaming trajectories. In: ICDE. pp. 186–197. IEEE (2012)
8. Xu, J., Lu, H., Bao, Z.: Imo: a toolbox for simulating and querying" infected" moving objects. PVLDB **13**(12), 2825–2828 (2020)
9. Zheng, K., Zheng, Y., Yuan, N.J., Shang, S., Zhou, X.: Online discovery of gathering patterns over trajectories. TKDE **26**(8), 1974–1988 (2013)

View publication stats